

Online Book store Project in SQL

Described:

The Online Bookstore project is a database-driven system that allows users to browse, search, and purchase books online. The database stores information about books, authors, customers, and orders. SQL is used to manage the data efficiently, including queries for searching books by title, author, or category, and handling customer orders.

Key Features:

- Books Table – stores details like ISBN, title, author, price, and stock.
- Customers Table – stores customer details such as name, email, and address.
- Orders Table – manages purchases, linking customers with books.
- SQL Queries – used for inserting, updating, deleting, and retrieving records (e.g., searching for books, checking stock, generating bills).

Benefits:

- Easy data management and retrieval.
- Automated order tracking.
- Efficient inventory control.

1st TABLE CREATE:

```
CREATE TABLE Books (  
    Book_ID SERIAL PRIMARY KEY,  
    Title VARCHAR(100),  
    Author VARCHAR(100),  
    Genre VARCHAR(50),  
    Published_Year INT,  
    Price NUMERIC(10, 2),  
    Stock INT  
);
```

```
-- RETRIEVE THE DATA
```

```
SELECT * FROM Books;
```

2nd TABLE CREATE:

```
DROP TABLE IF EXISTS customers;  
CREATE TABLE Customers (  
    Customer_ID SERIAL PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(100),  
    Phone VARCHAR(15),  
    City VARCHAR(50),  
    Country VARCHAR(150)  
);
```

-- RETRIEVE THE DATA

```
SELECT * FROM Customers;
```

-- 3rd TABLE CREATE

```
DROP TABLE IF EXISTS orders;  
CREATE TABLE Orders (  
    Order_ID SERIAL PRIMARY KEY,  
    Customer_ID INT REFERENCES Customers(Customer_ID),  
    Book_ID INT REFERENCES Books(Book_ID),  
    Order_Date DATE,  
    Quantity INT,  
    Total_Amount NUMERIC(10, 2)  
);
```

-- RETRIEVE THE DATA

```
SELECT * FROM Orders;
```

1) Retrieve all books in the "Fiction" genre

Query:

```
SELECT * FROM Books  
WHERE Genre ='Fiction';
```

2) Find books published after the year 1950

Query:

```
SELECT * FROM Books  
WHERE Published_year > '1950';
```

3) List all customers from the Canada

Query:

```
SELECT * FROM Customers  
WHERE Country ='Canada';
```

4) Show orders placed in November 2023

Query:

```
SELECT * FROM Orders  
WHERE Order_Date BETWEEN '2023-11-01' AND '2023-11-30';
```

5) Retrieve the total stock of books available

Query:

```
SELECT SUM(Stock) AS Total_Books FROM Books;
```

6) Find the details of the most expensive book

Query:

```
SELECT * FROM Books ORDER BY Price DESC  
LIMIT 1;
```

7) Show all customers who ordered more than 1 quantity of a book

Query:

```
SELECT * FROM Orders  
WHERE Quantity > 1;
```

8) Retrieve all orders where the total amount exceeds \$20

Query:

```
SELECT * FROM Orders  
WHERE Total_Amount > 20;
```

9) List all genres available in the Books table

Query:

```
SELECT DISTINCT genre FROM Books;
```

10) Find the book with the lowest stock

Query:

```
SELECT * FROM Books ORDER BY Stock ASC  
LIMIT 1;
```

11) Calculate the total revenue generated from all orders

Query:

```
SELECT SUM(Total_Amount) AS Revenue FROM Orders;
```

12) Retrieve the total number of books sold for each genre

Query:

```
SELECT b.Genre, SUM(o.Quantity) AS Total_Books_Sold
FROM Orders o
JOIN Books b ON o.book_id = b.book_id
GROUP BY b.Genre;
```

13) Find the average price of books in the "Fantasy" genre

Query:

```
SELECT AVG(price) AS Average_Price FROM Books
WHERE Genre ='Fantasy';
```

14) List customers who have placed at least 2 orders

Query:

```
SELECT o.customer_id, c.name, COUNT(o.Order_id) AS Order_Count FROM Orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY o.customer_id, c.name
HAVING COUNT(Order_id) >= 2;
```

15) Find the most frequently ordered book

Query:

```
SELECT o.Book_id, b.title, COUNT(o.order_id) AS Order_count FROM Orders o
JOIN Books b ON o.book_id = b.book_id
GROUP BY o.Book_id, b.title
ORDER BY Order_count DESC
LIMIT 1;
```

16) Show the top 3 most expensive books of 'Fantasy' Genre

Query:

```
SELECT * FROM Books
WHERE genre ="Fantasy"
ORDER BY Price DESC
LIMIT 3;
```

17) Retrieve the total quantity of books sold by each author

Query:

```
SELECT b.author, SUM(o.quantity) AS Total_Books_Sold
FROM Orders o
JOIN books b ON o.book_id = b.book_id
GROUP BY b.author;
```

18) List the cities where customers who spent over \$30 are located

Query:

```
SELECT DISTINCT c.city, total_amount FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.total_amount > 30;
```

19) Find the customer who spent the most on orders

Query:

```
SELECT c.customer_id, c.name, SUM(o.total_amount) AS Total_Spent
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY Total_Spent DESC
LIMIT 1;
```

20) Calculate the stock remaining after fulfilling all orders

Query:

```
SELECT b.book_id, b.title, b.stock, COALESCE(SUM(o.quantity),0) AS Order_Qty,
b.stock - COALESCE(SUM(o.quantity),0) AS Remaining_Qty
FROM Books b
LEFT JOIN orders o ON b.book_id = o.book_id
GROUP BY b.book_id;
```